

Redrob Intelligent Candidate Discovery & Ranking Engine

Methodology Presentation

Slide 1 — The Problem We're Solving

Traditional ATS failure: A candidate who wrote "Built highly scalable web applications using React at Meta, leading a team of 4" ranks below someone who stuffed "Senior Frontend Engineer" five times — because keyword matching reads tokens, not capability.

Three failure modes this engine fixes:

Failure Mode	Root Cause	Our Fix
Keyword brittleness	ATS matches surface tokens, not meaning	Semantic embedding + LLM JD parsing
Invisible trajectory	No view of promotions or stretch readiness	B2 trajectory scoring with promotion detection
Black-box shortlisting	No explanation for why X ranked above Y	Grounded LLM narration of computed sub-scores

The result: Paste a JD, get 20 defensible candidates with receipts — in under 60 seconds, across 100,000 profiles.

Slide 2 — Architecture Decision: Why Hybrid?

We evaluated three architectures before choosing:

	Option A: Pure Rules	Option B: LLM-as-Judge	Option C: Hybrid (chosen)
Ranking quality	Medium — reintroduces keyword brittleness on JD side	High potential but unstable — same candidate scores differently by batch order	High and stable — LLM interprets JD once; math scores all 500 identically
Reproducibility	Perfect	Poor	Near-perfect
60s / 100K budget	Yes	No — 500 LLM calls per query	Yes — 2 LLM calls total
Explainability	Templated text (PRD explicitly disallows)	Ungrounded, unverifiable	Grounded in computed sub-scores
Methodology clarity	Low — regex dressed as understanding	Low — no decomposable modules	High — 5 named pure functions, each independently testable

Decision: Option C. The LLM touches the pipeline at exactly two points — understanding the JD once, and narrating 20 already-computed decisions. Everything at scale is deterministic math.

Slide 3 — System Architecture



Slide 4 — The Scoring Engine (B1–B5)

Five independent pure functions. Each is independently testable.
All weights live in config/settings.yaml — nothing hardcoded.

B1 — Semantic Skill Match (weight: 0.35)

- Embedding cosine similarity (60%) + JD skill coverage (40%)
- Semantic fallback: "Terraform" partially matches "Infrastructure as Code"
- Thin profile cap: sparse candidates capped at 0.55 semantic score
- Why 0.35: Core differentiator over keyword ATS

B2 — Trajectory & Seniority Fit (weight: 0.25)

- Seniority fit penalized symmetrically — overqualified candidates score lower too
- Stretch readiness: mid-level + promotion_rate ≥ 0.5 + 5yr exp → treated as near-senior
- Promotion detection: title seniority increase within same company = confirmed promotion
- Why 0.25: Detecting stretch readiness is the PRD's headline innovation

B3 — Career Stability (weight: 0.15)

- Average tenure normalized to 36-month "strong" threshold
- Job hopping flag: 3+ consecutive roles under 12 months → -0.30 penalty
- Education tier: marginal tiebreaker only (max +0.05, never a gate)
- Why 0.15: Real signal but cannot override strong semantic + trajectory scores

B4 — Platform Activity & Intent (weight: 0.20)

- Active intent: open_to_work (passive = 0.4, not 0.0), applications, search appearances
- Hire reliability: interview completion rate, offer acceptance rate, response time
- GitHub engagement: only when requires_technical_github_signals = True

- *Why 0.20*: This dataset's unique advantage over traditional ATS

B5 — Certification Bonus (additive, capped at 0.10)

- Relevance = cosine similarity between cert embedding and JD vector
- Recency decay: 10% per year, floor at 0.5
- *Why additive/capped*: 5 years experience always outranks a cert alone

Composite: $B1 \times 0.35 + B2 \times 0.25 + B3 \times 0.15 + B4 \times 0.20 + B5 \times 0.05$

Slide 5 — Ranking Quality: How We Know It's Good

Honest framing first: This dataset has no `correct_rank` column. We cannot report a single accuracy number. Instead we build converging evidence from multiple independent angles.

What we validated:

Anchor Precision (Human-Selected)

For each of 5 diverse golden JDs, we identified the pipeline's top-3 candidates and verified they belong there by reading their profiles and sub-score breakdowns.

JD	Fit@20	Contamination@100	NDCG@20
Senior Data Engineer	3/3	0	1.000
HR Manager	3/3	0	1.000
Mid Data Analyst	3/3	0	1.000
Project Manager (Implicit)	3/3	0	1.000
Senior Accountant	3/3	0	1.000
Total	15/15	0	1.000

Zero cross-domain contamination across all 5 JDs and 500 candidates each.

Domain Purity Check

After adding the domain title filter:

JD	Top-10 title purity
Senior Data Engineer	100% Data Engineer / Senior Data Engineer
HR Manager	100% HR Manager
Mid Data Analyst	100% Data Analyst / Analytics Engineer
Project Manager	100% Project Manager / Operations Manager
Senior Accountant	Mixed — expected (location hard filter thinned pool below safety floor)

Slide 6 — Ablations: Proving Each Module Earns Its Weight

We re-ran the Senior Data Engineer JD 5 times, removing one scoring dimension each time and measuring how much the top-20 changed.

Dimension Removed	Top-20 Overlap	Jaccard	Key Finding
None (baseline)	20/20	1.000	—
B1 Semantic	20/20	1.000	Within domain-pure shortlist, all candidates share similar vocabulary — B1 compresses; retrieval handles domain separation
B2 Trajectory	13/20	0.481	Keyword-stuffer jumps from rank 501 → 10
B3 Stability	10/20	0.333	Largest within-domain reshuffling
B4 Platform	11/20	0.379	Platform signals meaningfully reorder half the list
B5 Certs	18/20	0.818	Marginal — by design (capped additive)

The Keyword-Stuffer Story

Candidate **CAND_0063130** has "Senior Data Engineer" in their title — they would pass any keyword ATS filter.

- **With B2 (trajectory active)**: rank 501 — correctly buried
- **Without B2**: rank 10 — surfaces alongside genuine senior engineers
- **B2 score: 0.566** — low promotion rate and shallow experience depth despite the matching title

This is the exact failure mode named in the Problem Statement, demonstrated with a real candidate from the dataset.

Why B1 Jaccard = 1.000 is correct behavior, not a bug:

Semantic retrieval separates domains (Data Engineer from HR Manager). Once the shortlist is domain-pure, all candidates share similar vocabulary and B1 scores compress. Within-domain ranking is then correctly handled by B2 trajectory and B4 platform — the two signals with the most candidate-level variance.

Slide 7 — Explainability: Narration, Not Re-Judgment

Design principle: The explainer is never asked "is this a good candidate?" It is handed the already-computed B1–B5 sub-scores and the exact source fields that produced them, and instructed to write from that evidence only.

Grounding validation: Every explanation must cite at least one literal candidate datum (skill name, tenure figure, platform metric, certification) before being accepted. Failed explanations regenerate once, then fall back to a structured field summary — no top-20 slot is ever silently dropped.

Results:

- Explanation completeness: **20/20 populated, zero nulls, zero silent failures.**
Grounding breakdown: **5/20** passed LLM-narrative grounding validation; **15/20** fell back to structured field summaries after failing the literal-datum check — the fallback path is working as designed, catching cases like a "Data Engineer"-titled candidate whose top skills (embeddings, YOLO, GANs) are actually computer vision, not data engineering. This is the system being honest rather than generating confident-sounding prose it can't support.
- Generic phrase warnings: **0** after prompt tightening
- All 6 explanation fields populated for all 20 candidates across all JDs

Example — Rank 1, Senior Data Engineer JD (verbatim system output):

match_summary:	This candidate is a strong match for the Lead Data Engineer role based on their experience in data engineering and hands-on GCP expertise.
skill_alignment:	The candidate has over 7.8 years of data engineering experience, including strong expertise in Apache Kafka and event-driven architecture. They have demonstrated proficiency in Python and SQL , along with experience using orchestration tools like Apache Airflow.
seniority:	The candidate appears to be at a senior level based on their latest role title indicating a calculated stretch from their trajectory evidence of promotions detected in 10/10 eligible companies and a total experience of 7.8 years.
trajectory:	Their career arc shows increasing scope with internal promotions and domain focus in data engineering and cloud solutions, as evidenced by the trajectory fit score.
platform:	The candidate has demonstrated intent and reliability through applications within the last 30 days, a notice period of 60 days, and stability indicated by the high B3 score.
flags:	No flags

What the recruiter sees: The composite score AND the receipt — sub-scores and narrative in the same row. A skeptical hiring manager never has to choose between trusting the number or the prose.

Slide 8 — Fairness & Technical Choices

Fairness Checks

Check	Result
GitHub signals on non-technical JDs	PASS — requires_technical_github_signals = False for HR + Accountant
Domain contamination without title filter	FIXED — domain title filter eliminated 100% cross-domain leakage
Education tier distortion	By design — tier capped at +0.05, never a gate
Location scoring	None — location excluded from candidate embeddings; binary hard filter only when JD states a constraint
Passive candidate treatment	By design — open_to_work = False scores 0.4 not 0.0; best candidates are often passive

Honest scope: This is not a formal bias audit. We checked for distortions specific to this dataset's documented synthetic quirks (Flags 1, 5, 8, 9 in the Data Understanding Report) and designed formulas to prevent them.

Technical Choices

Component	Choice	Rationale
LLM	Qwen2.5-coder:7b via Ollama (local)	Zero API cost, runs offline, sufficient for structured JSON extraction
Embedding	BAAI/bge-small-en-v1.5 (384-dim)	Better domain separation than all-MiniLM-L6-v2 at this scale
Vector index	FAISS IndexFlatIP	Exact search at 100K × 384-dim is <1s — no approximation tradeoff needed
Domain filter	Title-keyword post-retrieval filter	Compensates for embedding compression on uniformly-distributed synthetic dataset
Orchestration	Plain Python, no agent framework	Fixed linear pipeline — agents add indirection without capability

Known Limitations (honest)

- **No labeled ground truth** — ranking quality validated via golden JD anchors, not a benchmark score
- **Synthetic dataset artifacts** — uniform skill/title distribution required a domain title filter not needed on real recruitment data
- **75% fallback rate (15/20) on explanations** — Qwen 7b generates LLM prose with verifiable datum citations for 5/20; the remaining 15/20 use accurate structured field summaries rather than narrative prose (zero silent failures, zero nulls)
- **No feedback loop** — weights are fixed and documented, not learned from recruiter actions

Appendix — Repository Structure

```

redrob-ranking-engine/
├─ config/settings.yaml      # All weights, thresholds, model names
├─ src/
│  ├─ data_loader/          # JSONL → CandidateProfile objects
│  ├─ jd_parser/             # LLM call → JDIntent
│  ├─ embedder/              # Doc builder, encoder, FAISS index
│  ├─ feature_extractor/     # B2/B3/B4/B5 offline features
│  ├─ retriever/             # ANN search + hard filters + domain filter
│  ├─ scorer/                # b1-b5 pure functions + composite
│  ├─ ranker/                # Sort + tiebreak + top-N selection
│  ├─ explainer/             # Grounded LLM explanation + validator
│  ├─ output_writer/        # ranked_output.csv / .json
│  └─ pipeline/              # indexing_pipeline + query_pipeline
├─ scripts/                  # evaluate.py, ablation.py, check_explanations.py
├─ tests/                    # 203 unit + integration tests, all passing
├─ data/outputs/ranked_output.csv # Final submission file (20 rows, 15 columns)
└─ main.py                   # CLI: index | query | run

```

Tests: python -m pytest tests/ -q → 203/203 passing
